



JAVASCRIPT



INTRODUCTION

Langage JS

- Langage de la technologie web
- Langage impératif, interprété, procédurale et objet
- Script exécuté côté client
- Créé par Brendan Eich en mai 1995
- Standardisé en juin 1997 sous le nom d'ECMAScript**
- Dernière version ES11, juin 2020
- Attribuer des caractéristiques de mise en forme à des groupes d'éléments.



INTRODUCTION

- Principalement employé dans les pages web interactives mais aussi pour les serveurs avec l'utilisation (par exemple) de Node.js.
- Le langage JavaScript permet des pages web interactives, et à ce titre est une partie essentielle des applications web.
- La majorité des navigateurs web disposent d'un moteur JavaScript dédié pour l'interpréter, indépendamment des considérations de sécurité qui peuvent se poser le cas échéant.
- Du code JavaScript peut être intégré directement au sein des pages web, pour y être exécuté sur le poste client. C'est alors le navigateur web qui prend en charge l'exécution de ces programmes appelés scripts.



INTRODUCTION

Dans une page Web

Généralement, JavaScript sert à contrôler les données saisies dans des formulaires HTML, ou à interagir avec le document HTML via l'interface Document Object Model, fournie par le navigateur (on parle alors parfois de HTML dynamique ou DHTML). Il est aussi utilisé pour réaliser des applications dynamiques, des transitions, des animations ou manipuler des données réactives, à des fins ergonomiques ou cosmétiques.

Sur un serveur Web

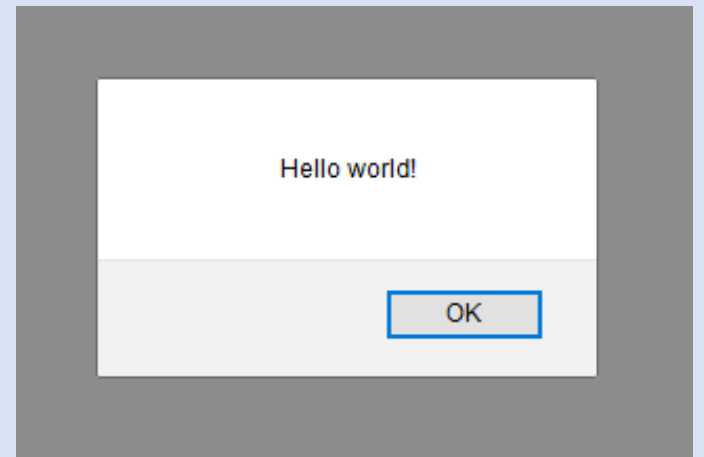
JavaScript peut également être utilisé comme langage de programmation sur un serveur HTTP à l'image des langages comme PHP, Python, ASP, etc. D'ailleurs le projet CommonJS travaille dans le but de spécifier un écosystème pour JavaScript en dehors du navigateur (par exemple sur le serveur ou pour les applications de bureau natives).

INTÉGRER UN SCRIPT JS A LA PAGE WEB

1ère méthode : écrire le JavaScript directement dans le code HTML

Pour écrire du JavaScript dans le code HTML, il suffit de l'écrire entre les balises `<script>` et `</script>`. Le code ci-dessous permet d'afficher le message « Hello World » à l'utilisateur grâce à la fonction `alert`.

```
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Hello World</title>
</head>
<body>
  <script>
    alert("Hello World !");
  </script>
</body>
</html>
```



INTÉGRER UN SCRIPT JS A LA PAGE WEB

2ème méthode (recommandée) : séparer le JavaScript du HTML

Il est préférable d'externaliser le code JS du code HTML, on devra donc utiliser deux fichiers séparés.

Le fichier HTML

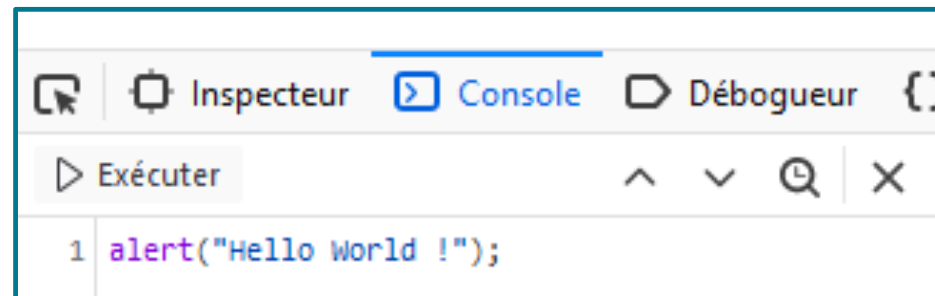
```
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Hello World</title>
</head>
<body>
  <!-- contenu HTML -->
  <script src="script.js"></script>
</body>
</html>
```

Le fichier script.js associé

```
alert("Hello World !");
```

COMMENT TESTER DU CODE JAVASCRIPT

- On peut bien entendu utiliser un **éditeur de texte** pour écrire le code HTML, le CSS et le code JavaScript. C'est d'ailleurs ce qu'il faut faire lorsque l'on a projet Web complet à développer.
- On peut également utiliser la **console JavaScript d'un navigateur**. Dans Firefox, on peut ouvrir la console Web par le raccourci Ctrl + Maj + K (ou dans Menu puis Développement web puis Console web). Il suffit alors d'écrire le code JS dans cette console puis l'exécuter.



- Enfin, on peut utiliser un **IDE en ligne** comme replit (<https://replit.com>) ou jsFiddle (<https://jsfiddle.net/>).

LA SYNTAXE EN JAVASCRIPT

JavaScript emprunte la plupart des éléments de sa syntaxe à Java, C et C++ mais sa syntaxe est également influencée par Awk, Perl et Python.

- JavaScript est **sensible à la casse** et utilise l'ensemble de caractères Unicode.
- Les **instructions** (appelées *statements*) **sont séparées par des points virgules**.
- Un **bloc d'instructions** est **délimité par une paire d'accolades** :

```
{  
    instruction_1;  
    instruction_2;  
    .  
    .  
    .  
    instruction_n;  
}
```


LA SYNTAXE EN JAVASCRIPT

Les noms des variables sont appelés **identifiants**.

La déclaration d'une variable est précédée de :

- **var** : La variable à une portée globale (connue dans tout le script)
- **let** : La variable à une portée locale (connue seulement dans le bloc courant).
- **const** : Une constante n'est connue que dans le bloc courant et n'est pas modifiable. Une constante est aussi appelée un littéral.

Il y a deux manières d'écrire un commentaire :

- Les commentaires sur une seule ligne se placent sur la fin de la ligne, après `//` (deux slashes).
- Les commentaires sur plusieurs lignes se placent entre `/*` et `*/`.

LA SYNTAXE EN JAVASCRIPT

Déclaration d'une variable : Une variable est typée dynamiquement. Cela signifie qu'il n'est pas nécessaire de spécifier le type de données d'une variable lors de sa déclaration. C'est lors de l'affectation que JavaScript type la donnée :

```
var variable = 42; // variable est un entier (Int)
variable = "Merci pour le dîner."; // variable est
une chaine de caractères

x = "La réponse est "+ 42; // "La réponse est 42"
y = 42+" est la réponse"; // "42 est la réponse"

"37" - 7; // 30

"37"+ 7; // "377"
```

LA SYNTAXE EN JAVASCRIPT

Une déclaration de fonction est construite avec le mot-clé `function`, suivi par :

- Le nom de la fonction.
- Une liste d'arguments à passer à la fonction, entre parenthèses et séparés par des virgules.
- Les instructions JavaScript définissant la fonction, entre accolades, `{ }`.

```
function carré(nombre) {  
    return nombre*nombre;  
}
```



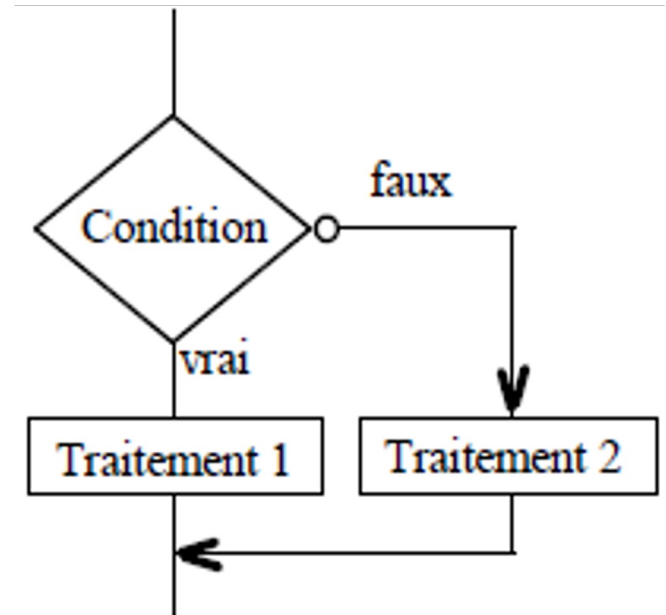
LA SYNTAXE EN JAVASCRIPT

L'instruction conditionnelle

```
if ( condition ) {  
  Traitement1();  
}  
else {  
  Traitement2();  
}
```

```
var age      = prompt('Quel âge avez-vous ? (en années)');  
var majeur  = (age >= 18);
```

```
if ( majeur )                // on effectue le test : majeur == true  
{  
    alert("Salut gamin !");  // action si condition vraie  
}  
else // SINON  
{  
    // action si condition fausse  
}
```



LA SYNTAXE EN JAVASCRIPT

switch case

```
var texte;
var jour = prompt("Quel jour est-il ?");

switch ( jour )    // on indique sur quelle variable on travaille
{
    case "lundi" :    // dans le cas où c'est le début de la semaine
    case "mardi" :    // on peut tester deux valeurs à la suite
        texte = "courage !!!";
        break;        // ne pas oublier le mot-clef break sinon le JS continue

    case "mercredi" : // dans le cas où c'est le début de la semaine
        texte = "c'est le jour des enfants";
        break;

    case "jeudi" :    // dans le cas où c'est la fin de la semaine
    case "vendredi" :
        texte = "bientôt le we !";

        break;

    default:          // il faut traiter les autres jours (cas par défaut)
        texte = "vive le week end !";
        break;
}

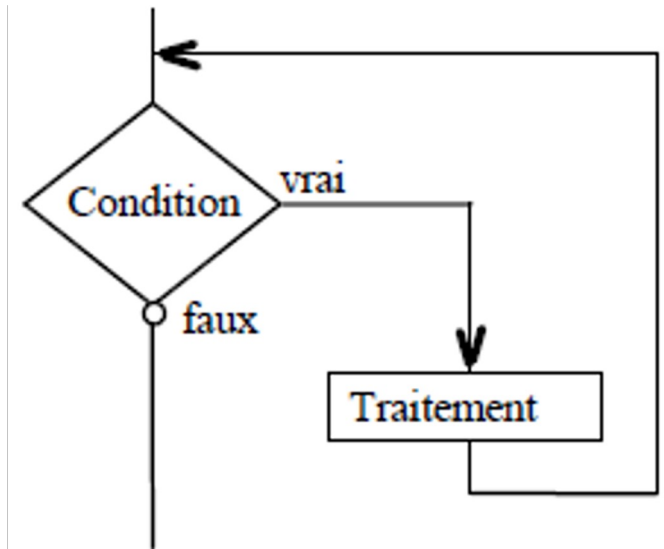
alert(texte);
```

LA SYNTAXE EN JAVASCRIPT

Boucle tant que (while) : nombre d'itérations inconnu

```
var i = 0; // initialisation
while (i < 10) // condition
{
    alert(i); // action

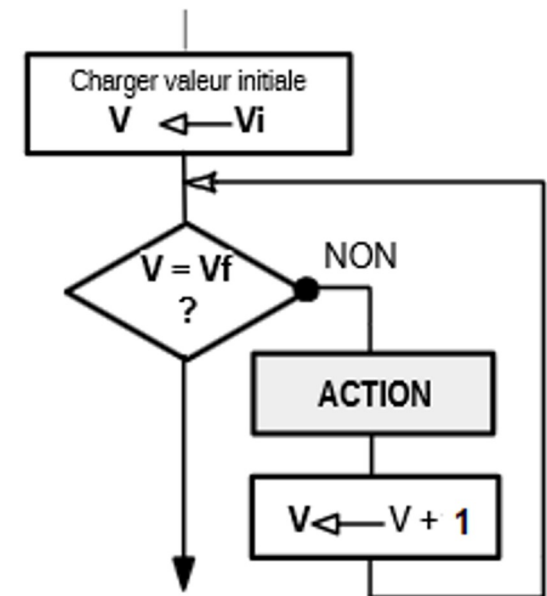
    i++; // incrémentation
}
```



LA SYNTAXE EN JAVASCRIPT

Boucle for : nombre d'itérations connu

```
for (var i=0; i<10; i++)  
    alert(i);
```



LA SYNTAXE EN JAVASCRIPT

Les tableaux

Les tableaux sont une suite de variables de même type, situées dans un espace contigu en mémoire. Un tableau a une dimension bien précise.

```
var table = new Array();
```

On peut initialiser un tableau de deux façon :

Élément par élément

```
var noms = new Array();  
noms[0] = "Pierre";  
noms[1] = "Paul";  
noms[2] = "Jacques";
```

À la déclaration

```
var noms = new Array("Pierre", "Paul", "Jacques");  
var best_scores = new Array(2.5, 4.7, 3.14);
```

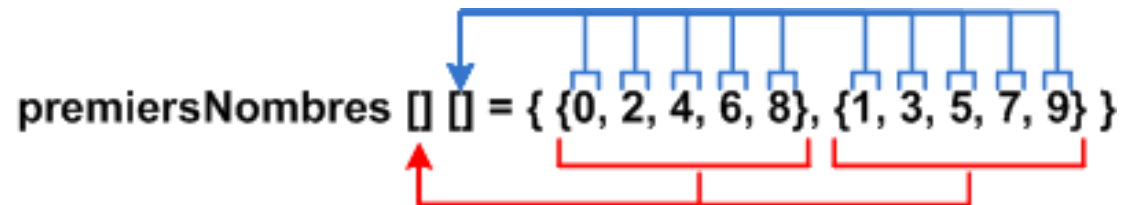

LA SYNTAXE EN JAVASCRIPT

Parcourir un tableau

```
var noms = new Array("Pierre", "Paul", "Jacques");  
noms.sort(); // on tri le tableau par ordre alphabétique
```

```
for (var i=0; i < noms.length; i++)  
{  
    chaine = "element : " + i + " -> " + noms[i];  
    alert(chaine);  
}
```

Tableaux multi-dimensionnels



Nous changeons de colonne par le biais de la première paire de crochets

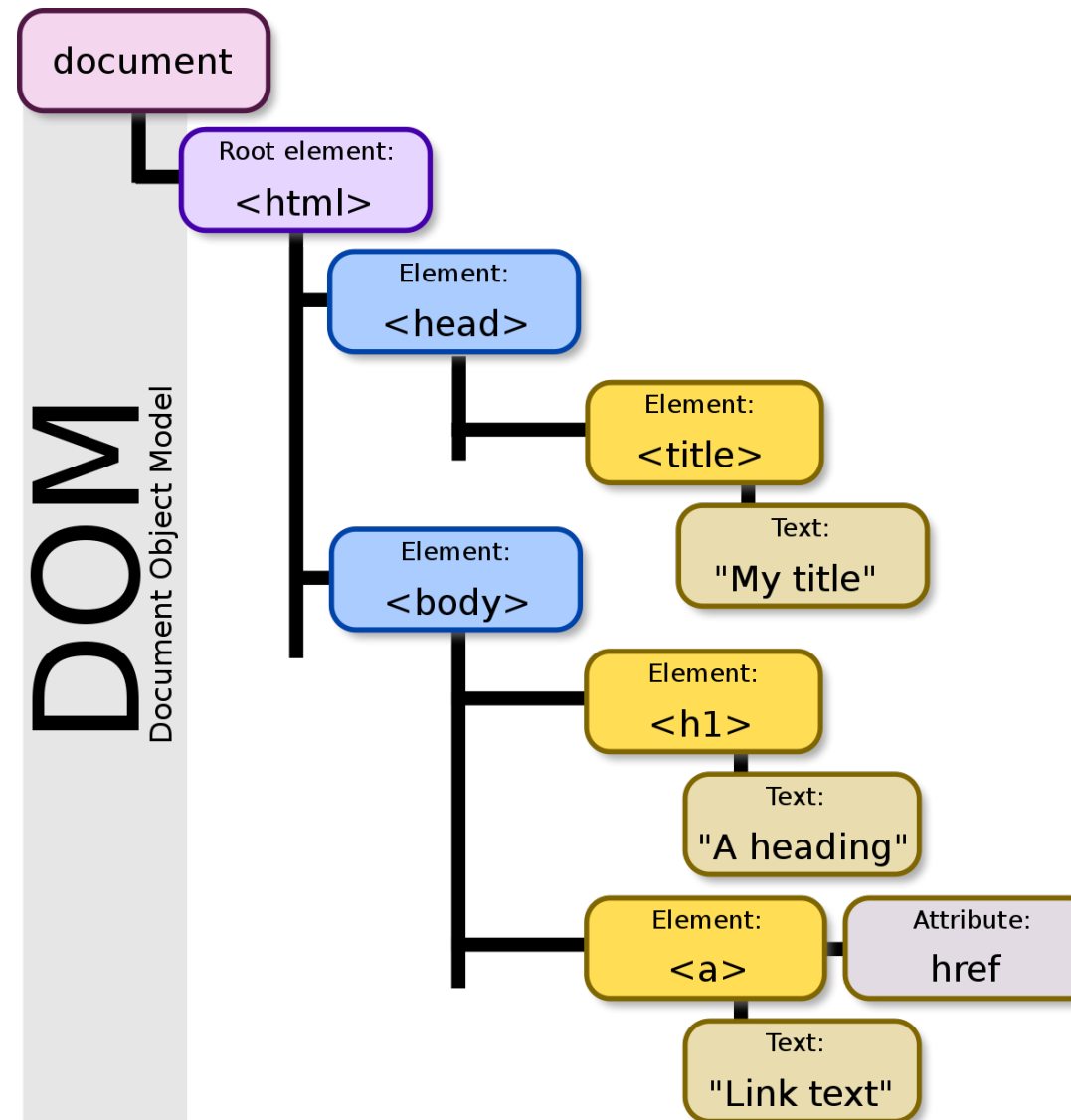
Nous choisissons le terme d'un tableau grâce à la deuxième paire de crochets

```
// on cree le tableau, contenant les lignes  
var grille = new Array();  
  
// on cree les lignes les unes après les autres  
for (var i=0; i<9; i++)  
    grille[i] = new Array();  
  
// on parcourt les lignes...  
for (var i=0; i<9; i++)  
    // ... et dans chaque ligne, on parcourt les cellules  
    for (var j=0; j<9; j++)  
        grille[i][j] = 0;
```

INTERACTION AVEC L'UTILISATEUR DANS UNE PAGE WEB

JAVASCRIPT DOM HTML

JavaScript peut accéder à tous les éléments d'une page Web en utilisant le modèle d'objet de document (DOM). En fait, le navigateur Web crée un DOM de la page Web lorsque la page est chargée. Le modèle DOM est créé sous la forme d'une arborescence d'objets.



INTERACTION AVEC L'UTILISATEUR DANS UNE PAGE WEB

JAVASCRIPT DOM HTML

Le code HTML DOM est une interface de programmation pour HTML. Il définit :

- Les éléments HTML comme des objets
- Les propriétés de tous les éléments HTML
- Les méthodes pour accéder à tous les éléments HTML
- Les événements pour tous les éléments HTML

En d'autres termes : Le code HTML DOM est une norme pour savoir comment **obtenir, modifier, ajouter ou supprimer** des éléments HTML.

INTERACTION AVEC L'UTILISATEUR DANS UNE PAGE WEB

JAVASCRIPT DOM HTML

L'interface de programmation est l'ensemble des **propriétés** et **méthodes** de chaque objet :

```
<html>
  <body>
    <p id="demo"></p>
    <script>
      document.getElementById("demo").innerHTML = "Hello World!";
    </script>
  </body>
</html>
```

Dans cet exemple :

- **document** : est un objet qui contient tous les éléments de la page web
- **getElementById()** : est une méthode qui permet d'accéder à l'élément indiqué en paramètre ("demo" dans l'exemple)
- **innerHTML** : est une propriété qui contient le code HTML de l'élément accédé

INTERACTION AVEC L'UTILISATEUR DANS UNE PAGE WEB

JAVASCRIPT DOM HTML

Trouver des éléments HTML

Méthode	Description
<code>document.getElementById("#id")</code>	Trouver un élément par son identifiant id
<code>document.getElementsByTagName("name")</code>	Trouver un élément par son type de balise (tag name)
<code>document.getElementsByClassName("name")</code>	Trouver un élément par le nom de la classe (class) qui lui est affecté
<code>document.querySelector("selecteur")</code>	retourne le premier élément dans le document correspondant au sélecteur

INTERACTION AVEC L'UTILISATEUR DANS UNE PAGE WEB

JAVASCRIPT DOM HTML

Changer le texte d'une balise : la propriété `innerHTML`

On peut changer le texte d'un élément HTML en utilisant la propriété `innerHTML` de cet élément.

```
function changeTexte() {  
var para = document.getElementById("#monPara");  
para.innerHTML = "Bonjour tout le monde !";  
}
```

Généralités sur JS

Bonjour tout le monde !

[Cliquez ici](#)

INTERACTION AVEC L'UTILISATEUR DANS UNE PAGE WEB

JAVASCRIPT DOM CSS

Pour changer le style d'un élément HTML, utilisez la syntaxe suivante :

`document.getElementById(id).style.property = new style`

```
<html>
  <body>
    <h1 id="h1">Hello World!</h1>
    <script>
      document.getElementById("h1").style.color = "blue";
    </script>
    <p>La couleur du titre a été changée par un script.</p>
  </body>
</html>
```

INTERACTION AVEC L'UTILISATEUR DANS UNE PAGE WEB

JAVASCRIPT DOM CSS : liste des propriétés CSS

CSS	JavaScript
background	background
background-color	backgroundColor
background-image	backgroundImage
background-position	backgroundPosition
background-repeat	backgroundRepeat
border	border
border-color	borderColor
border-width	borderWidth
color	color
display	display
float	cssFloat
font	font
font-family	fontFamily
font-size	fontSize

CSS	JavaScript
font-weight	fontWeight
height	height
left	left
margin	margin
overflow	overflow
padding	padding
position	position
text-align	textAlign
text-decoration	textDecoration
text-indent	textIndent
text-transform	textTransform
top	top
vertical-align	verticalAlign
visibility	visibility

INTERACTION AVEC L'UTILISATEUR DANS UNE PAGE WEB

JAVASCRIPT DOM CSS : Exemple

```
function changeStyles() {  
var para = document.getElementById("#monPara");  
var corps = document.getElementById("body");  
para.style.color="red";  
para.style.fontWeight="bold";  
corps.style.backgroundColor="rgb(255,255,0)";  
}
```

Généralités sur JS

Voici une page Web interactive grâce à JS.

Cliquez ici

INTERACTION AVEC L'UTILISATEUR DANS UNE PAGE WEB

LES EVENEMENTS

Les événements permettent de déclencher une fonction selon qu'une action s'est produite ou non. Ces événements peuvent être associés à n'importe quel élément HTML (un bouton `<button>`, un paragraphe `<p>`, un titre `<h.>`, un bloc `<div>` etc.).

Il existe beaucoup d'événements, en voici quelques-uns :

Événement	Description
click	Cliquer sur l'élément
dblclick	Double-cliquer sur l'élément
mouseover	Faire entrer le curseur sur l'élément
mouseout	Faire sortir le curseur de l'élément
keydown	Appuyer (sans relâcher) sur une touche de clavier sur l'élément
keyup	Relâcher une touche du clavier
change	Changer la valeur d'un champ de formulaire

INTERACTION AVEC L'UTILISATEUR DANS UNE PAGE WEB

LES EVENEMENTS

Exemple : On souhaite associer à un bouton, l'événement consistant à afficher le message « Hello World ! » dès lors que l'on clique sur ce bouton.

1ère méthode : Utiliser les attributs HTML du gestionnaire d'événements (très ancienne → OBSOLETE donc à ne plus utiliser car obsolète !)

Codes HTML et JS

```
<body>
  <button onclick="printMsg()">Clic</button>
  <script>
    function printMsg() {
      alert("Hello World !");
    }
  </script>
</body>
```

INTERACTION AVEC L'UTILISATEUR DANS UNE PAGE WEB

LES EVENEMENTS

2ème méthode : Utiliser les propriétés du gestionnaire d'événements (ancienne mais peut être utilisée)

Code HTML

```
<body>
  <button id='btn'>Clic</button>
  <script src="script.js"></script>
</body>
```

Code JS (fichier script.js)

```
var bouton = document.querySelector('#btn');

bouton.onclick = function() {
  alert("Hello World !");
}
```

3ème méthode : Utiliser la fonction `addEventListener()` (la plus récente donc celle à privilégier)

Code HTML

```
<body>
  <button id='btn'>Clic</button>
  <script src="script.js"></script>
</body>
```

Code JS (fichier script.js)

```
var bouton = document.querySelector('#btn');

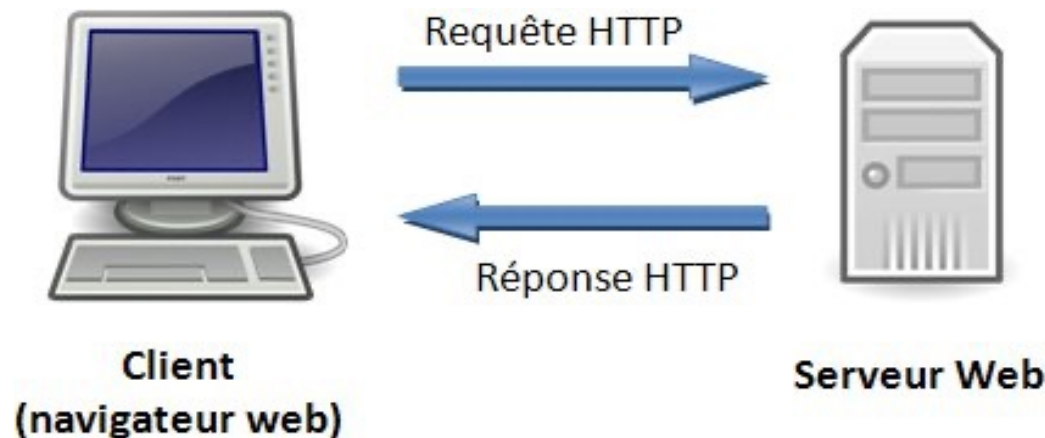
bouton.addEventListener('click', function() {
  alert("Hello World !");
})
```

INTERACTION AVEC L'UTILISATEUR DANS UNE PAGE WEB

LES FORMULAIRES

Sur internet, on retrouve une configuration Client-Serveur.

1. Le client demande à un serveur un service (requête HTTP)
2. Le serveur lui fournit (réponse HTTP)



Le client peut utiliser deux méthodes pour transmettre sa requête et ses paramètres.

- Méthode **GET** : les informations sont lisibles clairement via l'URL.
- Méthode **POST** : Les informations sont cachées dans l'entête de la requête http.

INTERACTION AVEC L'UTILISATEUR DANS UNE PAGE WEB

LES FORMULAIRES

La balise <form> définit un formulaire qui est utilisé pour collecter les informations d'utilisateur.

```
<form action="action_page.html" target="_blank"
method="GET">
```

...

```
</form>
```

- `action=` : L'action à exécuter lorsque le formulaire est soumis.
- `target=` : Si le résultat soumis ouvrira dans un nouvel onglet de navigateur, une nouvelle fenêtre ou la fenêtre actuelle.
- `method=` : La méthode HTTP (**GET** ou **POST**) à utiliser lors de l'envoi des données de formulaire

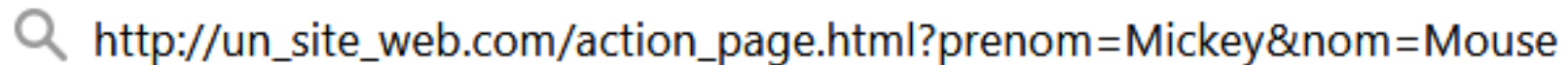
INTERACTION AVEC L'UTILISATEUR DANS UNE PAGE WEB

LES FORMULAIRE

Quand utiliser GET?

La méthode par défaut. Les données de formulaire soumis seront visibles dans le champ d'adresse de la page :

•Notes sur GET:



- ☐ La longueur d'une URL est limitée (2048 caractères)
- ☐ Ne jamais utiliser GET pour envoyer des données sensibles ! (Elles seraient visibles dans l'URL)

Quand utiliser POST?

Toujours utiliser POST si les données de formulaire contiennent des informations sensibles ou personnelles.

Notes sur POST:

- ☐ POST n'a pas de limite de taille, et peut être utilisé pour envoyer de grandes quantités de données.
- ☐ Les soumissions de formulaire avec POST ne peuvent pas être mis en favoris

INTERACTION AVEC L'UTILISATEUR DANS UNE PAGE WEB

LES ENTREES DE FORMULAIRE

Syntaxe	Description
Texte : <code><Input type = "text"></code> Texte :	Définit un champ de saisie de texte d'une ligne
Choix unique : <code><Input type = "radio"></code> Choix unique : ☐	Définit un bouton radio (pour sélectionner un élément dans une liste)
Choix multiple : <code><input type="checkbox"></code> Choix multiple : ☐	Définit une case à cocher (pour sélectionner plusieurs éléments)
0 <code><input type="range" value="50"></code> 100 0 <input type="range" value="50"/> 100	Définit un slider
a = <code><input type="number" value="50"></code> a = <input type="number" value="50"/>	Définit un nombre
Valider : <code><Input type = "submit"></code> Valider : <input type="submit" value="Envoyer"/>	Définit un bouton d'envoi (pour soumettre le formulaire)

INTERACTION AVEC L'UTILISATEUR DANS UNE PAGE WEB

LES ENTREES DE FORMULAIRE

Attributs spécifiques aux formulaires :

- **form** : le formulaire auquel appartient l'élément
- **name** : nom de l'objet (nom qui nous sert à désigner cet objet en JS)
- **type** : type de l'objet (button, select, textarea...)

Certains éléments possèdent ces attributs :

- **checked et defaultchecked** : valeur booléenne, true si la case est cochée (cochée par défaut dans le cas de defaultchecked), false sinon
- **disabled** : si cet attribut vaut true, l'objet est grisé (impossible à modifier / cliquer)
- **maxlength** : nombre de caractères maximal que peut contenir ce champ
- **readonly** : si cet attribut vaut true, l'utilisateur ne peut pas modifier le contenu du champ
- **size** : nombre de caractères / d'options qui sont affichés simultanément
- **value et defaultvalue** : valeur (valeur par défaut dans le cas de defaultvalue) de l'élément

INTERACTION AVEC L'UTILISATEUR DANS UNE PAGE WEB

L'élément HTML5 <output>

L'élément <output> représente un conteneur dans lequel on injecte le résultat d'un calcul opéré par un script.

Exemple

```
<form oninput="x.value=parseInt(a.value)+parseInt(b.value)">
  0
  <input type="range" id="a" name="a" value="50">
  100 +
  <input type="number" id="b" name="b" value="50">
  =
  <output name="x" for="a b"></output>
  <br><br>
  <input type="submit">
</form>
```

0  100 + = 135